

UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO

FELIPE LIMA ARAGÃO

RELATÓRIO MILESTONE 2 - JACKUT

MACEIÓ-AL
2026

SUMÁRIO

1 ESTRUTURA.....	3
2 FUNCIONALIDADES.....	4
MILESTONE 1.....	4
MILESTONE 2.....	4
3 PRINCIPAIS COMPONENTES E INTERAÇÕES.....	5
3.1 Diagrama (LINK).....	5
3.2 Main.....	5
3.3 Facade.....	5
3.4 Services.....	6
3.5 Repositories.....	7
3.6 Models.....	8
3.6 Exceptions.....	9
4 PADRÕES E ESCOLHAS DE DESIGN.....	9
4.1 Facade.....	9
4.2 Strategy.....	10
4.3 Observer.....	10
4.4 Service Layer.....	11
4.5 Repository.....	11
5 DESENVOLVIMENTO DA MILESTONE 2.....	12

JACKUT

1 INTRODUÇÃO

Este relatório apresenta o desenvolvimento do Jackut, uma rede de relacionamentos inspirada em sistemas como o Orkut, desenvolvida a partir das user stories e dos testes de aceitação fornecidos. O projeto tem como foco a implementação da lógica de negócio, sem interface gráfica, expondo suas funcionalidades por meio de uma Facade acessada pelos scripts do EasyAccept. No milestone 2, o sistema foi ampliado para ter comunidades, mensagens para comunidades, novos tipos de relacionamento entre usuários e remoção de contas. O projeto foi desenvolvido em java para a disciplina de Programação 2 (COMP372) 2026.1 - UFAL

2 ESTRUTURA

```
Jackut/  
+-- lib/  
|   +-- easyaccept.jar           - Biblioteca EasyAccept  
+-- tests/                       - Testes de aceitação  
+-- data/                        - Dados persistidos em XML  
+-- docs/                        - Documentação Javadoc gerada  
+-- src/br/ufal/ic/jackut/  
    +-- Main.java                - Ponto de entrada dos testes  
    +-- Facade.java              - Interface pública do sistema  
    +-- model/  
    |   +-- strategy/            - Estratégias de relacionamento  
    |   +-- observer/           - Observadores  
    +-- repository/              - Armazenamento e consulta de dados  
    +-- exception/               - Exceções de domínio
```

O Jackut foi separado em camadas para manter a lógica de negócio isolada da interface de testes:

Camada de Interface: *br.ufal.ic.jackut.Facade*. Expõe os métodos esperados pelo EasyAccept e delega as operações para os serviços responsáveis.

Camada de Regras de Negócio: *br.ufal.ic.jackut.service*. Contém os serviços que implementam as regras de usuários, sessões, relacionamentos, recados, mensagens, comunidades e remoção de usuários.

Camada de Modelo: *br.ufal.ic.jackut.model*. Representa as principais entidades do sistema: *Usuario*, *Sessao*, *Recado*, *Mensagem*, *Comunidade* e *ParticipacaoComunidade*.

Camada de Repositório e Persistência: *br.ufal.ic.jackut.repository*. Mantém os dados em memória e persiste usuários, comunidades e participações em XML usando *XMLEncoder* e *XMLDecoder*.

O ponto de entrada *Main* executa os scripts do EasyAccept, apontando para a *Facade* como classe de acesso ao sistema.

3 FUNCIONALIDADES

3.1 Milestone 1

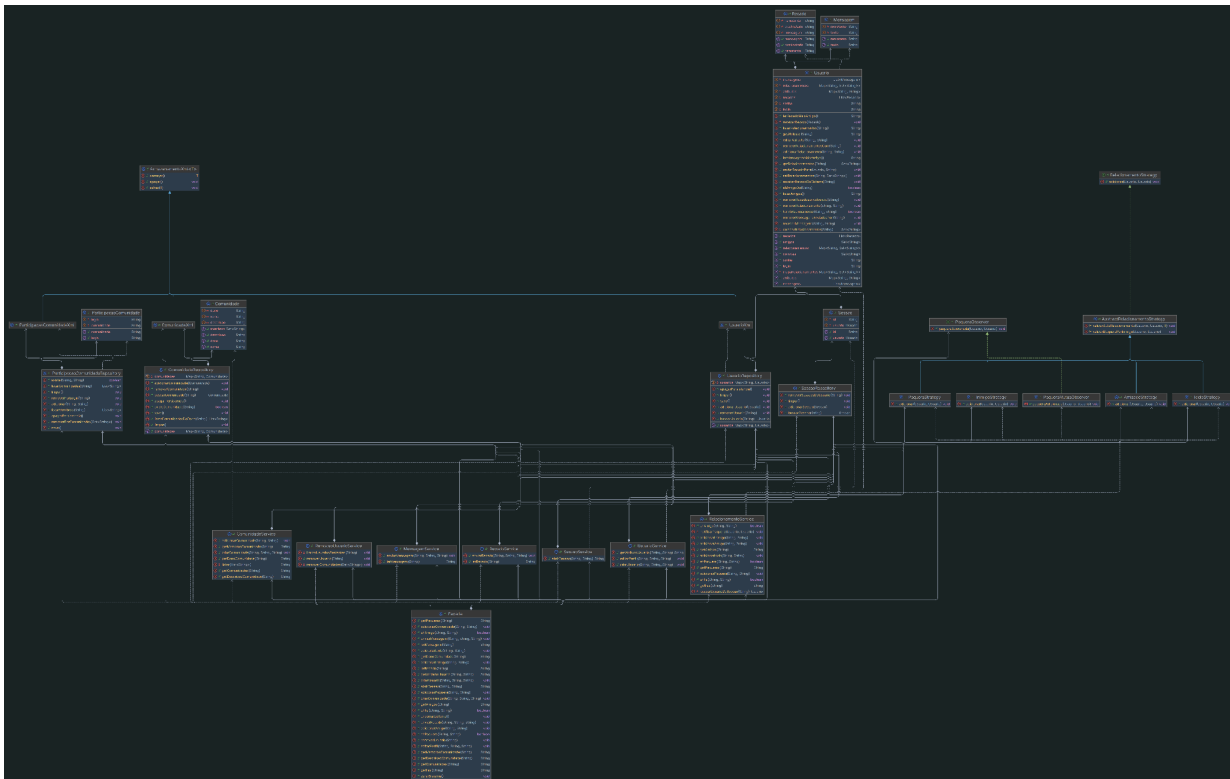
- Criação de conta
- Abertura de sessão
- Criação e edição de perfil
- Adição de amigos
- Envio e leitura de recados

3.2 Milestone 2

- Relacionamentos de ídolos, fãs, paqueras e inimigos.
- Notificação de paquera.
- Criação e participação em comunidades.
- Envio de mensagens para membros de comunidades.
- Remoção de usuários.

4 PRINCIPAIS COMPONENTES E INTERAÇÕES

4.1 Diagrama ([LINK](#))



Para melhor visualização do diagrama, acessar o link com a imagem em alta resolução

4.2 Main

Executa os testes de aceitação do EasyAccept.

4.3 Facade

A *Facade* instancia os repositórios e serviços usados pelo sistema:

- *UsuarioRepository*
- *SessaoRepository*
- *ComunidadeRepository*
- *ParticipacaoComunidadeRepository*
- *UsuarioService*
- *SessaoService*
- *RelacionamentoService*

- *RecadoService*
- *MensagemService*
- *ComunidadeService*
- *RemocaoUsuarioService*

Ela é a única classe que os testes precisam conhecer.

A *Facade* também controla o ciclo do sistema:

- **zerarSistema**: limpa a memória e remove os arquivos XML.
- **encerrarSistema**: salva os usuários e comunidades em XML.

4.4 Services

- **UsuarioService**
 - Criação de usuários.
 - Consulta de atributos de perfil.
 - Edição de atributos a partir de uma sessão válida.
- **SessaoService**
 - Autenticação de usuário.
 - Criação de sessões.

- **RelecionamentoService**
 - Relacionamentos de amizade, ídolos, fãs, paqueras e inimigos.
 - Notificação de paquera mútua usando observadores.
- **RecadoService**
 - Envio de recados entre usuários.
 - Leitura do recado mais antigo.
 - Bloqueio de recados inválidos
- **ComunidadeService**
 - Criação de comunidades.
 - Consulta de descrição, dono e membros.
 - Entrada de usuários em comunidades.
 - Listagem de comunidades de um usuário.
- **MensagemService**
 - Envio de mensagens para todos os membros de uma comunidade.
 - Leitura da mensagem mais antiga recebida por um usuário.
- **RemocaoUsuarioService**
 - Remoção de usuários.
 - Limpeza de vínculos do usuário removido em relacionamentos, recados, mensagens, sessões, comunidades e participações.

4.5 Repositories

- **UsuarioRepository**
 - Mantém os usuários em um *Map<String, Usuario>*.
 - Permite adicionar, buscar, remover e limpar usuários.
 - Encaminha operações de salvar, carregar e apagar dados para *UsuarioXml*.
- **SessaoRepository**
 - Mantém sessões abertas em memória.
 - Usa um *Map<String, Sessao>* para localizar sessões pelo id.

- Remove sessões associadas a um usuário
- ***ComunidadeRepository***
 - Mantém comunidades em um *Map<String, Comunidade>*.
 - Permite adicionar, buscar, remover, listar comunidades por dono e persistir dados.
 - Encaminha a persistência para *ComunidadeXml*.
- ***ParticipacaoComunidadeRepository***
 - Mantém os vínculos entre usuários e comunidades.
 - Lista membros de uma comunidade e comunidades de um usuário.
 - Encaminha a persistência para *ParticipacaoComunidadeXml*.
- ***ArmazenamentoXml***
 - Classe genérica para salvar, carregar e apagar dados XML dentro de *`data/`*.
 - É reutilizada por *`UsuarioXml`*, *`ComunidadeXml`* e *`ParticipacaoComunidadeXml`*.

4.6 Models

- ***Usuario***
 - Representa uma conta do Jackut.
 - Possui login, senha, atributos de perfil, relacionamentos, recados recebidos e mensagens recebidas.

- **Sessao**
 - Representa uma sessão aberta por um usuário autenticado.
 - Possui um id gerado por UUID e uma referência ao usuário da sessão.
- **Recado**
 - Representa um recado enviado entre usuários.
 - Possui remetente, destinatário e conteúdo da mensagem.
- **Mensagem**
 - Representa uma mensagem enviada para uma comunidade.
 - Possui remetente e texto.
- **Comunidade**
 - Representa uma comunidade do Jackut.
 - Possui dono, nome e descrição.
- **ParticipaçãoComunidade**
 - Representa o vínculo entre um usuário e uma comunidade.
 - Necessário para manter a ordem de entrada por usuário

4.6 Exceptions

As exceções, para representar erros específicos.

5 PADRÕES E ESCOLHAS DE DESIGN

5.1 Facade

O padrão Facade fornece uma interface única para as operações do sistema. Funcionando como uma "porta de entrada" para o sistema.

No projeto, esse padrão resolve a necessidade de fornecer uma API simples para os testes do EasyAccept.

Com isso, os testes não precisam instanciar serviços, repositórios ou modelos. Eles interagem apenas com a Facade, enquanto a implementação real fica distribuída internamente.

Exemplo:

```
public void criarUsuario(String login, String senha, String nome) no usages  👤 Felipe Aragão  
    throws ContaJaExisteException, SenhaInvalidaException, LoginInvalidoException {  
    usuarioService.criarUsuario(login, senha, nome);  
}
```

Nesse exemplo, a *Facade* recebe o comando externo e delega a criação de conta para o *UsuarioService*, que contém a regra de negócio.

5.2 Strategy

Os relacionamentos usam estratégias para isolar as regras de cada tipo de vínculo.

- *AmizadeStrategy*
- *IdoloStrategy*
- *PaqueraStrategy*
- *InimigoStrategy*

Todas seguem a interface *RelacionamentoStrategy*, com comportamento comum concentrado em *AbstractRelacionamentoStrategy*.

Esse padrão foi usado porque amizade, ídolo, paquera e inimigo têm a mesma intenção geral, adicionar um relacionamento, mas cada um possui validações e efeitos próprios. Com Strategy, cada regra fica em uma classe separada e o serviço não precisa concentrar todos os `ifs` desses casos.

5.3 Observer

O fluxo de paquera usa observadores para reagir a eventos do domínio. Quando uma paquera é adicionada, *RelacionamentoService* notifica os observadores registrados. O *PaqueraMutuaObserver* identifica paquera mútua e envia um recado do sistema aos usuários envolvidos.

Esse padrão foi usado porque a paquera mútua é uma consequência de uma ação, não a ação principal. Com Observer, o serviço adiciona a paquera e delega a reação para uma classe própria, deixando o fluxo mais simples e facilitando a inclusão de possíveis novas reações no futuro, caso fosse um projeto real.

5.4 Service Layer

O projeto usa uma camada de serviços para concentrar as regras de negócio. Essa escolha evita que a *Facade* ou os modelos acumulem responsabilidades de diferentes partes.

Cada serviço possui uma responsabilidade clara:

- *UsuarioService* cuida de cadastro e perfil.
- *SessaoService* cuida de autenticação.
- *RecadoService* cuida da troca de mensagens.
- *RelacionamentoService* cuida de amizades, ídolos, fãs, paqueras e inimigos.
- *RecadoService* cuida da troca de recados.
- *ComunidadeService* cuida de comunidades e seus membros.
- *MensagemService* cuida da troca de mensagens em comunidades.
- *RemocaoUsuarioService* cuida da exclusão de usuários e da limpeza dos dados associados.

Essa divisão facilita a manutenção, compreensão do projeto e reutilização de código, porque novas user stories podem ser adicionadas em serviços próprios ou em serviços já relacionados ao domínio da funcionalidade.

5.5 Repository

Os repositórios isolam como os dados são guardados e recuperados. As classes de serviço não precisam manipular diretamente arquivos XML nem conhecer detalhes de persistência.

Essa escolha também auxilia na manutenção e entendimento do código e projeto.

UsuarioRepository oferece operações de consulta e alteração de usuários, enquanto *UsuarioXml* fica responsável pelo armazenamento em arquivo.

SessaoRepository mantém sessões apenas em memória.

6 DESENVOLVIMENTO DA MILESTONE 2

Para a implementação das stories 5 a 9, a estratégia principal foi evoluir a estrutura existente com base nos feedback recebidos e mantendo a estrutura geral do projeto. Algumas regras que antes exigiam manipulação direta de coleções nos serviços passaram a ficar mais em *Usuario*, como adicionar, consultar, etc, ao mesmo tempo, as regras de negócio maiores foram mantidas nos serviços específicos. Isso foi feito para seguir a proposta do feedback sobre encapsulamento.

Para a US6, em vez de guardar os membros dentro de Comunidade, como era o plano inicial, foi criada a estrutura de *ParticipacaoComunidade*, com repositório e persistência próprios. Essa escolha foi feita para não haver duplicação de dados em usuário e comunidade, o que seria necessário para manter a ordem de entrada de cada usuário. Dessa forma só existe uma fonte de verdade no projeto, e os testes, que necessitam da ordem exata individual de entrada em cada comunidade são aceitos.

Para os novos relacionamentos, a principal decisão foi usar Strategy. Amizade, ídolo/fa, paquera e inimizade seguem a mesma ideia geral de criar um vínculo entre usuários, mas cada tipo possui validações e efeitos próprios. Separar essas regras em estratégias evita que *RelacionamentoService* vire uma classe cheia de “ifs” e “elses” e facilita a manutenção de cada um. No caso da paquera mútua, foi usado Observer para tratar o envio automático dos recados do sistema como uma consequência da ação principal, e não como uma regra misturada diretamente no fluxo de adicionar paquera.

Por fim, a remoção de usuário foi concentrada em *RemocaoUsuarioService* porque ela afeta várias partes do sistema ao mesmo tempo: usuários, sessões, relacionamentos, recados, mensagens, comunidades e participações. Colocar essa regra em outro lugar iria adicionar tarefas a mais nessas classes, tarefas essas que não deveriam ser responsabilidade da classe. Com um serviço específico, a operação de excluir o usuário fica em um ponto específico, mantendo a organização geral do projeto.